

Artificial Intelligence and Software Engineering

David Barstow

Schlumberger-Doll Research
Ridgefield, Connecticut

Introduction

This paper is based on a talk titled “Artificial Intelligence and Software Engineering.” That’s actually a bit broader than I’m going to be. In particular, I’m only going to look at attempts to apply AI techniques to software engineering problems. So I will specifically not discuss indirect contributions from AI to software engineering, of which there are many. I also won’t discuss the application of software engineering techniques when you’re building AI systems, of which there is a great need.

There are three parts to this paper. First, I’m going to give an overview of software engineering, because it’s important to understand what software engineering is, and I think many of us have been remarkably ignorant about it. Then I’ll look at the state of the art in research on the application of AI to software engineering. Finally, I’ll look at what the impact of all this might be on the practice of software engineering.

Part I

It's conventional to divide software engineering activities into two categories, usually referred to as programming-in-the-small and programming-in-the-large [DeRemer and Kron, 1976]. Programming-in-the-small is typically done by individuals or very small groups. A typical project might be a few thousand lines long, and it's typically not more than a few months of effort. Programming-in-the-large is done by very large groups of people, and programs have several hundred thousand or millions of lines, with very long intended lifetimes. Now these are obviously ends of a spectrum, but the issues involved in these two different categories are really quite different. So it's a useful distinction to try to make. Let me illustrate both of these by considering two problems that you, as a software engineer, might get involved with.

1.1 Programming-in-the-Small

The first example is related to Schlumberger's activities in oil well logging (Figure 1a). Basically, our service involves lowering an instrument down an oil well, pulling it back out, making measurements along the way, and then interpreting those measurements in terms of the things that our clients, oil companies, are interested in. For example, where is the oil? So, imagine that a tool designer has just built a new tool and you've been asked to write the software for it. Let's think about what you would do.

The first thing you do is try to write a specification for that software, some kind of complete and precise description of what the software is supposed to do (Figure 1b). As suggested by the bubbles in the figure, while you're doing this, you need to know something about the tool and about the physics upon which the tool's sensors are based.

Now, the next thing you do is realize that it will be a moderately big program, so you want to reduce it into more manageable pieces, as shown in Figure 1c. While you're doing this, you obviously need to know about the specification. You also need to know about the architecture of the machine you're going to run the software on, and you need to know the programming techniques that you learned when you were in school. You also find that you have to talk to the tool designer because, despite everybody's best attempts, the specification is neither complete nor precise.

Once you've got the little pieces, you try to implement them (Figure 1d). This again requires knowledge of the architecture of the target machine and programming techniques. And once again, you need to talk to the tool designer. Then comes testing (Figure 1e).

The goal here is to ensure that the source code actually implements what the specification says it ought to implement. This involves taking some data,

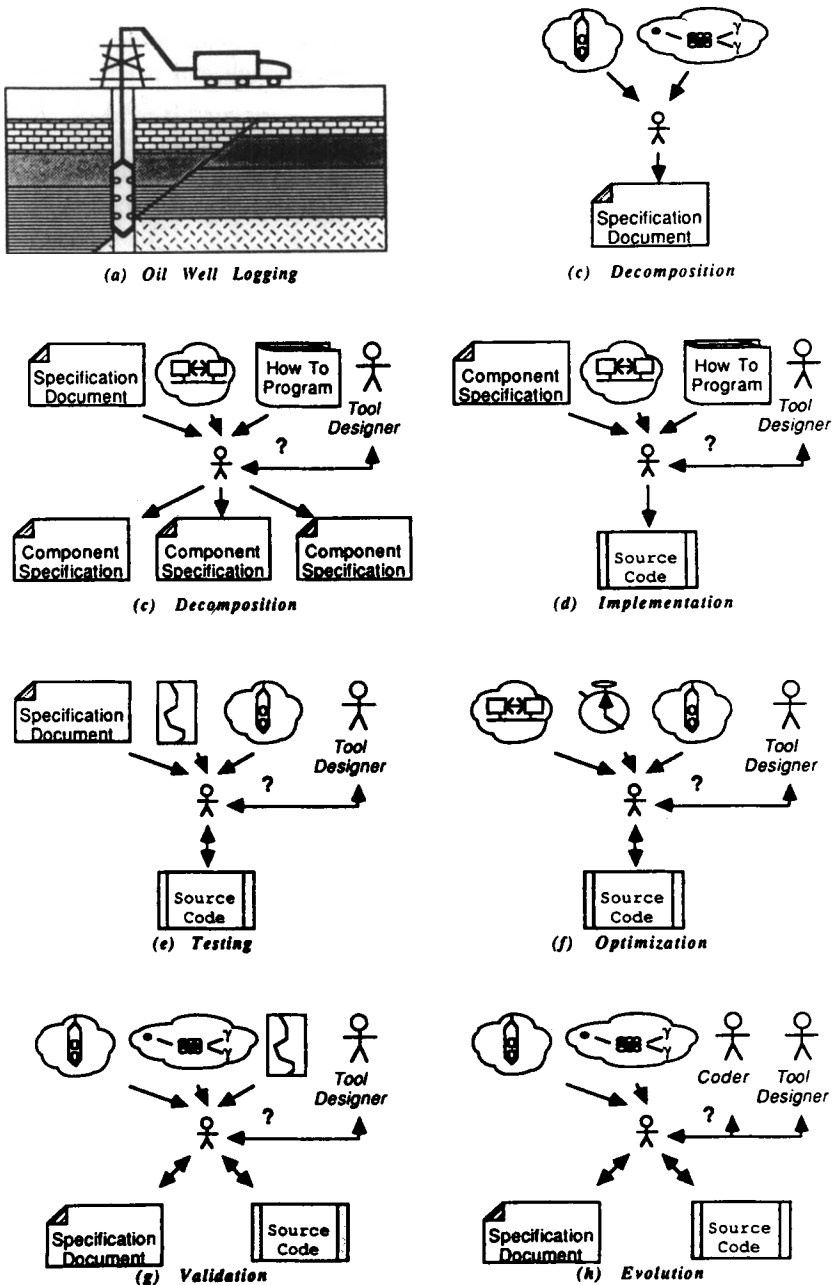


Figure 1 Programming-in-the-small.

studying the tool again, and talking with the tool designer to make sure you've covered all the cases. And in the process, of course, you change the code.

Next you find out that the code isn't fast enough and needs to be optimized (Figure 1f). There are some very tight constraints on the software for logging, and you find out that the software doesn't meet those. So you get out your stopwatch, study the architecture of the target machine, study the tool again, talk to the tool designer again, and finally try to get the code to be fast enough.

Then you ask yourself whether this really implements what the tool designer wants (Figure 1g). As you recall, all you've tested so far is that it matches the specification. Validation is the process of making sure that the specification specifies the software that the tool designer wants, that it satisfies the tool designer's real needs. In order to do this, you obviously need to make measurements, you need to study the tool and the physics, and you need to talk to the tool designer.

Now, you may think you're done, but in fact you're not, because someone has been busy designing a new tool, or perhaps developing a better understanding of the physics. So you have to rewrite the code to satisfy the needs of the new tool or the new understanding of physics (Figure 1h). One of the things you find is that you're not the same person you were when you wrote the code in the first place. So you have to talk to the coder who did it the first time as well as the tool designer. When you talk to the coder, you need to understand what decisions were made when he or she wrote the software. And not only what decisions were made, but why they were made. That is, you need to know the rationale for the decisions.

1.2 *Programming-in-the-Large*

For an example of programming-in-the-large, imagine you work for the Internal Revenue Service, and Congress passes a new tax law (Figure 2a). You've been told you have to write the software to process the tax returns. I don't know about you, but first I would panic. But once that panic is over, you realize that the first thing to do is figure out what's going on in the tax law. That is, you have to do requirements analysis (Figure 2b). Now, let's suppose there are 1800 pages of "legalese." There are no doubt numerous ambiguities, probably intentional, adding to what is already a rather difficult problem for you, so you talk to a tax expert to help you figure it all out, and you also hire a number of people to work with you.

Now, once you think you understand the tax law, the next thing you have to do is break the system down into components that can be considered or attacked as problems of programming-in-the-small (Figure 2c). That's usually called "design." In order to do this, it turns out you need to know some software engineering techniques, and you find again that you're talking to a tax expert.

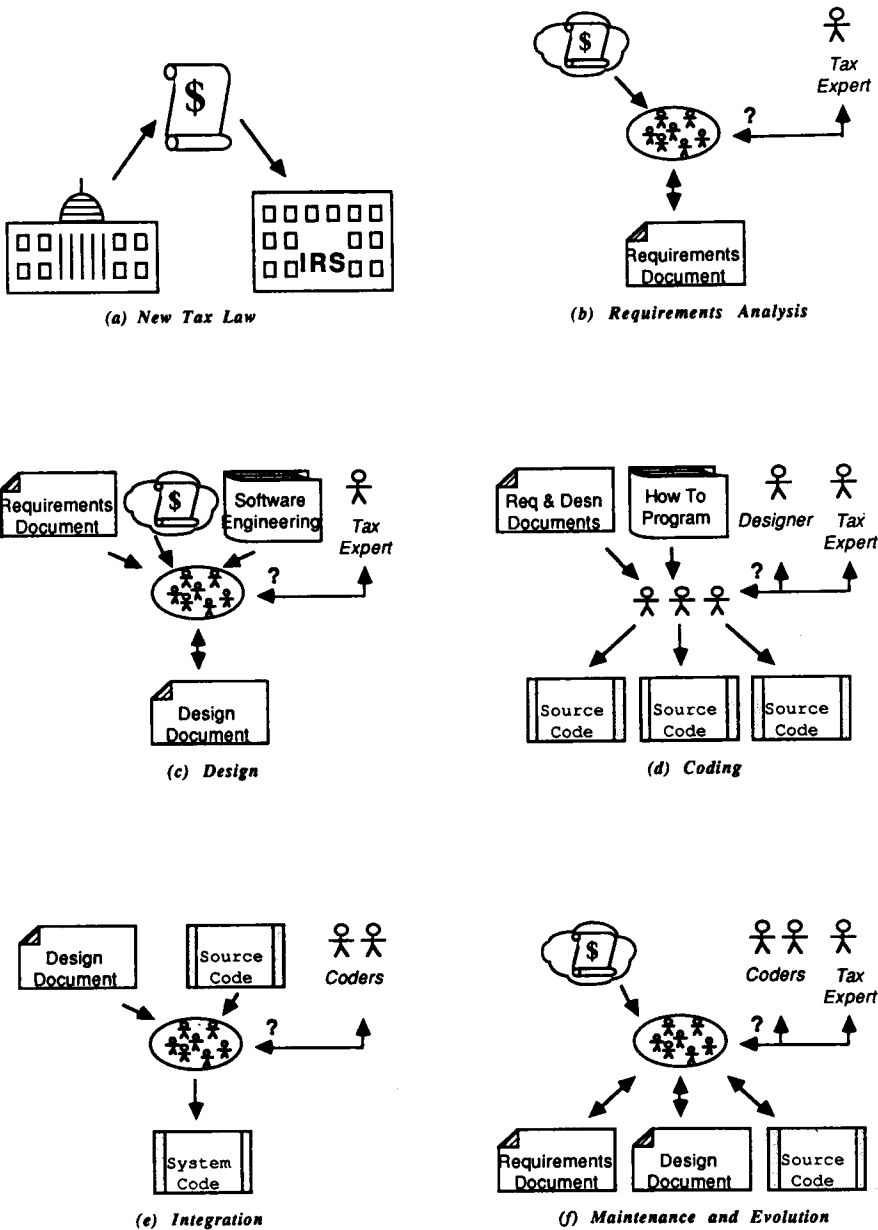


Figure 2

Now, having done the design, you go through the coding process (Figure 2d). This typically involves smaller groups of people, and it can be thought of as programming-in-the-small, for relatively independent activities. In doing this, of course, you need to know programming techniques, and you need to talk to the tax expert, but you also need to talk to the system designers, in order to understand why the system design is the way it is, in order to understand the nature of the interfaces of your piece of code with the other pieces of code.

Now, having completed the coding, you go through an integration process, again a group activity (Figure 2e). You get everything together to make sure you have the right large system. The integrators may not be the same as the coders, so the integrators have to talk to the coders.

Now, you have the same case here that you had with tool software (Figure 2f). Congress is already talking about changing the tax law. So while you've been working away at it, the requirements are changing. So you have to change the software that you've written. And in doing this, you have to talk not only to tax experts, but also to the coders and the designers. Because once again you're a different group than when you started. Once again you have to know not only what decisions were made, but also the reasons that they were made.

1.3 Distribution of Effort In Software Projects

Let's try to analyze these examples a little bit. First, it's important to understand the distribution of effort among the various activities. The pie chart in Figure 3a, based loosely on Barry Boehm's Cocomo model [Boehm, 1981], shows the relative amount of time spent in different activities involved in programming-in-the-small. (I've actually left off evolution; we'll see that in the chart for programming-in-the-large.) Notice that only a fifth of the time is spent in implementation, what we normally think of as programming. And only a little over half of the time is spent with code. Overall, the effort is pretty evenly distributed.

The distribution is not so even for programming-in-the-large (Figure 3b). In particular, according to the Cocomo model, an extremely small part of the time goes into coding. Most of the time goes into maintenance and evolution, something like 60 or 70 percent of the effort on a large software system. In fact, more recent studies seem to show that the number is increasing.

1.4 Knowledge Used In Software Activities

The next thing to look at is the knowledge used during these activities. We can see five general categories. Three obvious categories are software engineering methodologies, programming techniques, and the architecture of the target

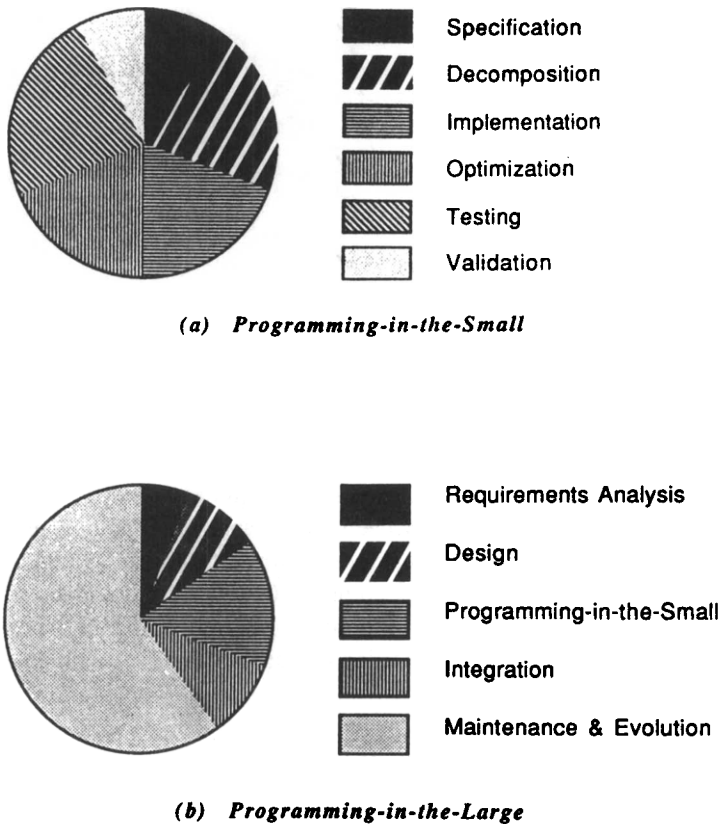


Figure 3 Distribution of effort in software projects.

machine. The other two are, I think, more interesting. One is the amount that you have to know about the application domain; the other is the history of the target software. Let's consider these in a little more detail.

	<i>Used</i>
<i>Programming-in-the-Small</i>	
Specification	✓
Decomposition	
Implementation	✓
Testing	✓
Validation	✓
<i>Programming-in-the-Large</i>	
Requirements Analysis	✓
Design	✓
Integration	
Maintenance and Evolution	✓

Figure 4 Knowledge of the application domain.

Figure 4 shows the various software activities, with check marks indicating activities which require a substantial amount of knowledge about the application domain. (Recall the little pictures of tools, physics, and taxation and the figures representing tool designers and tax experts.) Domain knowledge is used in two ways. First, you must have a rather deep understanding of the domain in order to specify and validate the software, where you want to make sure that you are doing the right thing. Similarly, a deep understanding of the domain is needed during requirements analysis and evolution. Second, you must use domain knowledge to help make implementation decisions during the implementation, testing, and design activities.

	<i>Created</i>	<i>Used</i>
<i>Programming-in-the-Small</i>		
Specification	✓	
Decomposition	✓	
Implementation	✓	
Testing		✓
Optimization	✓	✓
Validation		✓
<i>Programming-in-the-Large</i>		
Requirements Analysis	✓	
Design	✓	
Integration	✓	✓
Maintenance and Evolution	✓	✓

Figure 5 Knowledge of the target software.

Now, let's look at knowledge of the target software, that is, knowledge of what decisions were made earlier in the development process. It is interesting that, whereas knowledge of the domain exists *a priori*, knowledge of the target software is created during the process of writing the software. Figure 5 has check marks indicating where a significant number of decisions are made, and those are the places where that knowledge comes into existence. The second set of check marks indicates where that knowledge is used. Perhaps the most important mark is the one next to maintenance and evolution, because knowledge created throughout the earlier phases is relied on to a very great degree during maintenance and evolution.

1.5 Techniques for Knowledge Management

In fact, I would make the following assertion: *The high cost of software development and evolution is primarily due to the ineffectiveness of current techniques for managing knowledge about the application domain and about the implementation history.*

Let's look at what those current techniques are. They're listed in Figure 6, roughly in increasing order of frequency of use. Sometimes we have requirements documents, more frequently we have design or specification documents, and people usually put some comments in the code. But most of the time we rely on human memory and the wastebasket. That is, we think we will remember the decisions we made when we look at the code later, and we may just throw away any thoughts that we had about it, believing that they won't matter.

The weaknesses of these techniques are obvious. First, most of the information is not recorded. And in fact, that's one of the reasons for the great cost of the later phases of software engineering activities. Second, complex interactions are not made explicit. That is, you may make a design decision early on in the software development process that has ramifications throughout the code. And if all you have are the comments to look at, then you're not going to understand the nature of those interactions. And finally, of course, the information is difficult to access. If it was thrown away or put in somebody's memory, it's obviously very difficult to retrieve.

- Requirements Documents
- Design Documents
- Specification Documents
- Comments in Code
- Human Memory
- Wastebasket

Figure 6 Current techniques for knowledge management.

In other words, software engineering is fraught with uncertainty: A major source of problems during software development is uncertainty about requirements; a major source of problems during software evolution is uncertainty about design and implementation decisions made during development.

1.6 The Need for AI Techniques

So these are, to my mind, the fundamental problems faced by software engineering, and obviously, I believe that AI techniques ought to be able to come to the rescue here. A straightforward argument says that: (a) Effective knowledge management requires computer support and (b) Computer support for knowledge management requires AI techniques. Now, when I say the same kinds of things to software engineering audiences, this is the basic argument that I feel like I have to make. I would hope that I don't have to make this argument here, but in case *you* have to make it some time, that's the argument.

Now, it's certainly not the case that only AI is necessary, and we can mention many other computer technologies that are absolutely required to solve the software engineering problem, including at least data bases, communication systems, and user interfaces. But I think the argument for the necessity of AI is quite strong.

1.7 Review

Let's review what we've established so far. Software engineering activities are knowledge intensive, especially requiring knowledge of the application domain and of the target software. Many software engineering costs are due to the ineffectiveness of current techniques for managing the knowledge. And AI techniques ought to be able to help manage the knowledge more effectively.

Part II

Let's see what's been done in trying to apply AI techniques to software engineering activities. In doing this, I will look at several different paradigms, the results of the work to date, and the directions that I think we ought to be pursuing.

2.1 Deductive Synthesis

The first paradigm is usually referred to as deductive synthesis, and essentially it relies on an analogy between programming and theorem proving. In particular, there's an analogy between a specification and a theorem, and an analogy

between a program and the proof of the corresponding theorem. So the idea is that, in a mechanical way, you can go from a specification to a theorem, then you run your theorem prover forward to get a proof, and from the structure of the proof you go back to the program.

Let me just summarize what some of the basic techniques are. Typically, a specification will consist of two parts, a pre-condition and a post-condition. A pre-condition is usually a predicate on the input variables, say $P(X)$. A post-condition is a predicate on the input and output variables, say $Q(X, Y)$. The essence is that the software writer can assume that $P(X)$ holds of the inputs and must write software that guarantees that $Q(X, Y)$ is satisfied after the program halts. This specification corresponds to the theorem $\forall X \exists Y P(X) \Rightarrow Q(X, Y)$, and if we have a proof of this theorem, we can take the structure of the proof and map it into a program. For example, a constructive proof typically corresponds to an assignment statement, a case analysis corresponds to a conditional, and induction corresponds to recursion. There is, of course, a lot of detail in these techniques; I'm just trying to summarize them here.

The history of the technique goes back to two Ph.D. theses [Green, 1969; Waldinger, 1969]. Both came up with essentially the same observation, the correspondence between programs and proofs. Around 1980, Manna and Waldinger developed a new formulation [Manna and Waldinger, 1980]. It's fundamentally the same technique, but it's a new formulation that makes it a little bit easier to work with. They call it the tableau method. An interesting recent development is some work by Smith, looking at decomposition strategies and derived pre-conditions, with the goal of getting to some relatively more complicated kinds of algorithms, such as divide-and-conquer algorithms [Smith, 1985]. The state of the art is that there have been some demonstrations for some very simple programs, things like sorting, greatest common divisor, and binary search [Manna and Waldinger, 1985]. Those are really pretty simple programs.

So what are the major issues? One issue relates to size and complexity: How do you write larger and more complex programs? The work of Smith, for example, is aimed in that direction, but has not gotten very far. A second issue relates to the efficiency of the resulting program. And to understand this issue, you have to go back to the basic technique, where you're doing all of the work in the theorem-proving domain. So if you want an optimal program, then you have to make sure that you find the proof that corresponds to that optimal program. Typically, optimal programs are longer than the average program that someone will write. And so typically, the proof will be longer than the proof for the average program that you want to write. And therefore, you're trying to guide the theorem prover to find a long proof instead of a short proof, and most theorem provers are not oriented in that direction. So this is, I think, a fundamental weakness for this paradigm.

2.2 Program Verification

The second paradigm, program verification, is also based on theorem-proving techniques. The essence of it is that, from a specification and from a program, you can derive in a mechanical way some things called verification conditions. Now, if you can show that those verification conditions hold, that is, if you can run your theorem prover and find a proof, then that guarantees that the program satisfies the specification.

Let me illustrate the technique with an example. Suppose the specification's pre-condition is $P(X)$ and the post-condition is $Q(X, Y)$, and the program is:

```
if R(X) then Y ← F(X) else Y ← G(X)
```

Then the verification condition is

$$P(X) \Rightarrow (R(X) \Rightarrow Q(X, F(X))) \wedge (\neg R(X) \Rightarrow Q(X, G(X)))$$

You can see the correspondences among the parts of the pre-condition, post-condition, program, and verification condition. Of course, verification conditions for larger programs are much larger than this simple example.

This all goes back fundamentally to work in the 1960s on the semantics of programming languages by people like Floyd, Hoare, and Dijkstra [Floyd, 1967; Hoare, 1969; Dijkstra, 1976]. The first program verifier that I know about was by King [1971]. In the 1970s, there were a very large number of rather substantial verification projects, at places like Stanford, SRI and ISI. In the 1980s, we've had some successful demonstrations on small but nonetheless real-life programs, for example, encryption algorithms and certain kinds of secure operating systems. So the state of the art here is that we've had demonstrations for non-trivial algorithms, certainly more complicated algorithms than was the case in deductive synthesis, and also some other kinds of small programs and work on abstract data types. All of these have required a substantial amount of effort, both by the user and by the theorem prover. So it's certainly not a straightforward task to force one of these proofs to go through. But people have succeeded in doing it.

One major issue with program verification is size: The larger the program, the larger the verification condition, hence the more difficult it is to find the proof. Another issue is that writing specifications, even after you have the code, can turn out to be a rather difficult problem.

2.3 Transformational Implementation

Now, the third paradigm is one that is usually referred to as transformational implementation. The basic idea here is that you start with the specification and

transform it through a potentially very large number of very little steps, the end result of which is the program that satisfies the specifications. Now, with this paradigm, the techniques are not as clearly understood as they are in the case of verification and deductive synthesis. But we can identify certain categories of things that you have to do if you want to build a system like this.

One such thing to have is a wide-spectrum language, a term coined by Bauer [Bauer et al., 1978]. The basic idea here is that you need some coherent framework for representing the program during the entire process of going from a specification down to the code at the bottom. And since there are different concepts that appear in the specifications than appear in the code, you need to span a range of concepts, and that's why it's called a wide-spectrum language. Some of the concepts that you need to span here include abstract and concrete data types and operators, and different kinds of computational paradigms.

You also need a library of transformations. This is, after all, the mechanism to get from one step to the next in the transformation process. Some of the kinds of transformations that have been worked on include data type and operator refinement and global reformulations. A typical library may have many hundreds of transformations, so building the library is, itself, a major piece of work.

The other problem that can happen here is that, as you're going through the transformational process, there may be several different transformations that can be applied at the same time. So in fact, you have a search tree. Each of the different paths down the tree leads to a different program, and choosing one path over another amounts to making an implementation decision. The potential for explosive growth in that tree is very clear, and the trees can get extremely large, especially when you have thousands of steps. So search control is clearly important. People have looked at several techniques. One involves efficiency analysis—analyzing an intermediate-level program and guessing what the efficiency is going to be like. Heuristic techniques can be used to simplify or eliminate the need for analysis. However, user interaction is the most common technique to date; that is, the user decides which transformation to apply at any given point. One interesting technique is the idea of replay. That is, once you've gone through a sequence of transformations and you know which transformations were applied to which parts of the program, if you change the specification a little bit, you can replay the same sequence of transformations, or something close to it, and thereby save a lot of the work of thinking about which transformations to apply to those steps.

The history of the technique goes back to the early 1970s, when there were a number of transformation projects at places like Munich [Bauer et al., 1982], Stanford [Green, 1976], ISI [Balzer, 1981], Harvard [Cheatham et al., 1979], and elsewhere [Partsch and Steinbruggen, 1983]. In 1978, I used transformations as a way of representing programming knowledge, a way of encoding

knowledge about programming techniques [Barstow, 1979]. At about the same time, Kant did some work on using efficiency considerations to guide the search [Kant, 1983; Kant and Barstow, 1981]. Both of these were done as part of the PSI project at Stanford. Around 1982 or so, a couple of results came out of the Information Sciences Institute work that I think are worth noting. One is due to Fickas who attempted to use heuristics to guide the search [Fickas, 1985a]; another involved some work by Wile on describing transformation histories, essentially laying the basis for the replay idea [Wile, 1983].

So what's the state of the art? There's routine use of such systems in research settings, such as ISI [Balzer, 1985] and the Kestrel Institute [Smith et al., 1985]. And there are some attempts to demonstrate practical value for the transformational technology, including some commercial ventures that have every incentive to make the technology work. So I think one of the nice results that we can expect soon is a demonstration of practical value.

Now, what are the major issues? First, how do you want to specify things? All the projects use different techniques for specification, so I don't think there's any general agreement on that [Balzer et al., 1983; Balzer et al., 1978; Smith et al., 1985]. Second, search control: As I mentioned, there are a variety of approaches, and it does seem to be an issue that we have to address, and we really don't know how yet. And third, the implementation history: You need to record that implementation history somehow, and we only are beginning to understand ways to do that.

2.4 Programmer's Assistant

The fourth paradigm is usually called the programmer's assistant. The underlying idea comes from the observation that a human programmer has a great deal of knowledge about programming techniques. If you could build a system that shared that kind of knowledge, then the system ought to be able to help the programmer in the process of developing a program.

One of the basic techniques involves representing programs. You want to represent what you know about programs, not at the level of syntax of the target language, but rather at a somewhat more abstract level. Some of the techniques that have been developed include data flow, control flow, and hierarchical decomposition. Just as with transformations, you need a library of these things, usually called plans, and it's very similar to having a library of transformations. You also need a mechanism to do some kind of analysis. The basic idea here is that you do a data and control flow analysis on the program and then try to match that against what you have in the library. And the problem is that it won't always match, but it'll be close, so you have to transform things, either the plan or the program you're looking at, in order to find the right kind of match.

The history of this basically goes back to the Programmer's Apprentice project begun at MIT in the 1970s and described in a paper by Rich and Shrobe which is still worth reading [Rich and Shrobe, 1978]. Rich's thesis described the first plan representation system [Rich, 1981]. Waters, in the same project, did some work on integrating a conventional EMACS editor with a library of plans for doing some synthesis and analysis [Waters, 1985]. In a related effort, Johnson built a program that used similar techniques to analyze student programs to make suggestions to help the students learn about programming [Johnson and Soloway, 1985]. Now, the state of the art is that we've had some demonstrations of the use of plans for analysis and synthesis. To my knowledge, the size of the programs that have been analyzed are typically a few lines, perhaps a page or two. Some of the ones that have been synthesized using plans may go up to half-a-dozen pages.

So what are the major issues? I think it has turned out that analysis is much more difficult than had been hoped originally when Rich and Shrobe made their first proposal. And I think this difficulty is a very major issue, and it may turn out to be the stopper for this. Recognition is part of analysis, and some good matching techniques have been developed, but it's not clear whether that can carry us far enough to do analysis. And then, as with all the other paradigms, there's the question of large programs. All these systems work to one degree or another on programs that range from a few lines to a few pages. So the question here, as with the others, is what happens when you look at larger programs.

2.5 Other Work

Now, I want to cite several other pieces of work. I don't have time to go into them in detail, but there have been a variety of efforts in other ways of applying AI techniques to software engineering. These include alternative specification techniques, such as natural language [Heidorn, 1976] and examples [Smith, 1984]. Some work has been done at ISI on monitoring change and evolution in software [Balzer, 1987]. Swartout did some work on explaining software [Swartout, 1983]. Kant and Steier did some work on analyzing protocols for designing algorithms [Kant, 1985; Steier and Kant, 1985]. Greenspan and Fickas have both done some work on trying to use knowledge representation techniques in requirements analysis for programming-in-the-large. [Borgida et al., 1985; Fickas, 1985b]. And there has also been some work on data structure selection [Katz and Zimmerman, 1981]. There are other projects that I've left out, I just wanted to suggest some of the things that have been done. The reason I've picked four paradigms to describe in a little more detail is that I think they're the ones that have had the most attention. Because of that, we can say there really are a paradigms that have been explored.

2.6 Practical Utility

Now, one of the things that I'm afraid we have not seen is a demonstration of the utility of AI techniques to support software engineering in practical situations. I wish I were wrong in saying this, but I don't believe I am. The closest I think we have come are those few programs that have been verified with very great effort, and there are suggestions that some of the commercial work may be panning out. But I don't think we've yet had any really solid demonstrations of the practicality of all this. Now, I'd like to see if we can explore why: If there have been 20 years or so of work on this, why haven't we demonstrated anything practical?

I think one of the reasons is that it really is a hard problem. To see that, let's try to compare software engineering with medicine, an area in which AI seems to have had more success. Now, I'm *not* trying to compare software engineers and doctors in terms of their intelligence or skills. Rather, I want to look at the characteristics of the tasks that these people are performing, and Figure 7 shows five dimensions for comparison. The check marks indicate that the particular domain seems to have a greater amount or a greater diversity of knowledge.

In the first dimension, accumulated expertise, there have been something like two orders of magnitude more time spent on medicine than on software engineering, something like 2,000 years to 20 years. So there's certainly a greater amount of accumulated expertise in medicine. But that may not make it a harder task, because one result of that accumulated expertise is a set of useful abstractions. Doctors have nice short words that they can use to refer to things, and I think that we in software engineering haven't really developed those necessary abstractions yet. So although the check says that medicine ought to be harder, it is kind of double-edged.

The second dimension is complexity of subject, and it certainly seems to me that people are more complicated than any of the software systems that anybody's been able to build so far. I think medicine has a clear edge there.

On the other hand, we can look at the third dimension, variation of subjects. Depending on how you count, there are one or two varieties of people. However, if you look at software systems, there is enormous diversity. There are real-time systems, data base systems, communication systems, data analysis systems, and so on. So this great diversity means that you can't just teach a person all about software engineering and expect it to apply in all situations. So diversity is a major problem for software engineering.

The fourth dimension is the run-time environment. Your doctor may ask you whether people smoke where you work, but he's not likely to ask you much more about the environment in which you "run." And yet, when we build software systems, those systems have to interact with the world, and we have to study that interaction very, very carefully, or else the software isn't going to

	<i>Medicine</i>	<i>Software Engineering</i>
Accumulated Expertise	✓	
Complexity of Subjects	✓	
Variation in Subjects		✓
Runtime Environment		✓
Implementation History		✓

Figure 7 Amount and diversity of knowledge.

be correct. So a great deal more attention has to be paid to the run time environment in the case of software.

The final dimension is the implementation history. It's really not necessary for doctors to know why we have two hands or why we've evolved the way we have. But in the case of software, especially for maintenance, it's very important to know what decisions were made and why they were made. So that is another area in which software engineering at least appears to be somewhat more complicated than medicine.

So the amount and diversity of knowledge is one reason that I think that Software Engineering really is a very hard problem. So maybe it's not so bad that we haven't demonstrated practical successes yet.

But I think there's another reason, one for which we may bear more responsibility. And that's that most of the research on AI applied to software engineering has been relatively narrowly focused, missing what I think are many issues of practical importance. The focus, if you add up everything that's been done, has largely been on algorithm design, and a little more on data structure selection. But it turns out that if you look at how various kinds of effort are distributed, algorithm design and data structure selection are really only a small part of programming. And programming in fact is only a small part of software engineering. So it's as if we're solving a very small part of a very small part of the problem. The end result is that, if we successfully automated away algorithm design and data structure selection, there would be only a minimal impact on software engineering. To see this, let's look back at our two example software projects.

Figure 8a shows various activities involved in producing software for logging tools. The dots indicate major time-consuming activities and the checks indicate activities which make significant use of algorithms and data structures. You spend most of your time studying the tool and the physics, testing and validating the functionality, measuring performance, and looking at existing programs that you've written or that other people have written. Algorithm design will help you in writing signal processing algorithms and in writing code for computations and input and output. Notice the mismatch—most of the effort in AI applied to software engineering has been aimed at helping in ways that are relatively inconsequential in this particular category of software.

- Studying tool and physics
- Studying target machine and communication system
- ✓ Writing signal processing algorithms
- ✓ Writing code for computations and input/output
- Testing and validating functionality
- Measuring performance
- Studying existing algorithms

8a Logging tool software.

- Studying tax law
- Designing overall system structure
- ✓ Designing data base and communications systems
- ✓ Writing code for calculations and input/output
- Testing components and complete system
- Studying existing programs
- Communicating with colleagues

8b Tax Return software.

Figure 8 Importance of algorithms and data structures. The dots indicate major time-consuming activities and the checks indicate activities which make significant use of algorithms and data structures.

We can say the same thing about writing tax software (Figure 8b). A very large part of your time is spent studying the tax law, communicating with your colleagues, looking at other programs, and testing the various components of the complete system. Algorithm design and data structure selection are going to help a bit in designing data bases and communication systems, and in writing code for calculations and input and output. So once again, we see that the bulk of the effort has gone into activities that are relatively inconsequential.

2.7 Major Research Issues

So, if that's the case, what *are* the consequential research issues? For programming-in-the-small, I'd like to suggest three. One is the representation of domain knowledge. You remember the number of times, in Figures 1 and 2, that there was a little bubble that showed something about the domain in those various activities. I think the best plan of attack is to build several systems that are domain specific and that actually work in practical situations, and then to get down to the fundamental issue here, which is, how do you represent domain knowledge, for use by a programming or software engineering system, in a way that's domain independent? The second issue relates to techniques for controlling search, especially in the transformational paradigm. We have had a

little bit of progress on those, but the space is so huge here that it's a very critical issue. The third issue involves representing the implementation history. How do you represent, in some kind of useful way, the design decisions, the implementation decisions that have led from a specification down to a program? Now, the early transformational systems represented that as a sequence of transformations. But a sequence of a thousand transformations is not a very useful way to represent that information. There has to be more structure to it. So the question is, what kind of structure can we impose on that kind of representation? Now, I was being critical a few minutes ago about the application of AI to software engineering. In fact, all these issues are being addressed in one way or another, so the situation isn't quite as bad as completely ignoring the major issues.

On the other hand, it unfortunately is the case that the research issues related to *programming-in-the-large* have been almost totally ignored. So here are some issues that are ripe for working on and you don't have to worry about someone having solved them already. First, the representation of domain knowledge: Again, just as in the case of *programming-in-the-small*, it's a major issue. Here the question is, can you develop techniques to represent what you know about a domain as a result of doing a requirements analysis? There's been a little work on some relatively small examples, but it's certainly a major issue that we ought to be able to resolve, or that we ought to at least attack. Second, what are good techniques for describing large systems? At present we have a few techniques, such as the plan representation, for describing small programs. Can we find reasonable techniques for representing large systems and the interactions among the various components, in some kind of manageable way?

A third and related issue is, can we represent, again in a manageable way, the design history of large systems? This is to me the single most important problem. So if you want to hit something, if you want to solve a problem with high impact, this is the one I would suggest you look at. How do you represent the design history of very large systems? And I have no idea, no clue at all, about how to go about it. I hope one of you can come up with a clue. Finally, there are models of collaborative work. Especially in *programming-in-the-large*, you spend a lot of your time just talking with people. So, to the degree that we can develop models of collaborative work, that ought to help in the area of *programming-in-the-large*, but I don't see it as being substantially different from models of collaborative work in other areas [Association for Computing Machinery, 1986; Stefik et al., 1987].

2.8 Practical Experimentation

In order to achieve practical results, one guideline in addressing these issues is to build experimental systems in practical situations. If there was a failing in

the first decade of work on AI applied to software engineering, it was that we weren't following this guideline. We thought programming was writing little algorithms, like they taught us in school. That's not what programming and software engineering are all about. So we really have to look at realistic, practical situations, in order to make sure that we're addressing the right sorts of issues. So here are a few suggestions about the kinds of systems that I think are ripe for attacking. Domain specific automatic programming systems: I'll give you an example of one of those in a minute. Very high level language compilers: The REFINE¹ system being marketed by Reasoning Systems is an example based on the transformational implementation paradigm [*REFINE Users Guide*, 1985]. Program library manager: Software reuse is a hot topic these days in discussions of software productivity. One of the problems in reuse is, if you have a large library, how do you find the piece that you want to reuse? So, a manager that would help you find your way through that library would certainly be a valuable thing. Data structure selection advisor: A couple of data structure selectors have been built, but they haven't really been tested in practical situations, so I think that's ripe for doing. Project management advisor: There is certainly a lot of expertise that project managers have, and that ought to be able to be captured. Finally, a maintenance advisor for a specific system: It's important to notice that qualification for a specific system. I've heard people suggest that you ought to be able to build a maintenance advisor. After all, if maintenance and evolution is a major cost area, and you have people who do maintenance and evolution, then you ought to be able to build an advisor, using standard rule-based expert systems techniques. I think that's folly, at least to think that you could build one to work in general. Because in fact, a lot of the knowledge that the good maintainers and evolvers have is knowledge about the details of the specific system that they are maintaining and evolving. But that suggests that perhaps you could build one for a very specific system. So find a system that you think is going to have a long lifetime, a system that comes with some people who are good at maintaining and evolving it, and try to codify their expertise.

2.9 Φ NIX

Now, just to illustrate one of these, I want to describe a project that we're working on at Schlumberger-Doll Research. It's a project we call Φ NIX, a domain-specific automatic programming system for software that controls and records data from oil well logging tools [Barstow, 1985a, 1985b]. In fact, that's the very first example I showed you earlier. There are two basic goals. One is to enable a user, and you can think of a tool designer here, to describe a

¹ REFINE is a trademark of Reasoning Systems, Inc., Palo Alto, California.

problem in terms that are natural to him or her. The other goal is automatically to write software that is robust and efficient enough for routine use, because we want to put the software out on a truck and log a well with it. And that, I think, is a very valuable attribute of our efforts, because it gives us a kind of reality check.

Now, it turns out that logging software is rather interesting computationally. It's a mixture of value- and behavior-oriented computations. There are real-time constraints, due both to the communications systems and the physics. These are non-terminating programs. Most of the formal definitions of automatic programming say that the specifications are going to be satisfied when the program halts. If the program halts in our case, the specifications are without doubt not satisfied. There are concurrent constructs in our target architecture, so we get a chance to look at a bit of parallelism. And it's clearly a case of programming-in-the-small. This, at least, gives us a little bit of hope that we might be able to have some success, I think. And one of the other things that I like about logging software is that it's a special case of a more general class of software, namely device-control software.

There are five basic features to our approach. First, we're separating the total process into a formalization process and an implementation process, a rather standard type of separation. The formalization process we're imagining to be interactive, with the system possessing a significant amount of knowledge about the domain, in particular, knowledge about the tool in the form of a model of the device.

We specify software by what we call *stream expressions*. We model the interaction of the software with its external environment in terms of temporal sequences of values called *streams*, and a specification is a statement of constraints on the input and output streams. We're using a transformational approach with search control based on a certain kind of performance measure. In our case, it's not a measure of the absolute cost of something, because these programs are supposed to run forever, so their absolute cost ought to be infinite. Rather, we're looking at a measure of the load that they put on the processor as the guideline for search control.

That's about all I'm going to say about Φ NIX here. We haven't logged a well yet, so we certainly haven't demonstrated practicality, but I'm hoping that some time in the not-too-distant future, we will be able to do that.

2.10 Review

Let's see what ground we've covered in Part II. I think the important thing here is to see that research on AI applied to software engineering has had virtually no demonstrations of practical value. And it's important for us to understand why. In part it's because software engineering is a harder problem, I think, than others that have been addressed somewhat more successfully with

AI techniques, and in part because the research has been narrowly focused. I think that is changing. We're beginning to understand what we have to look at in order to have a practical impact on software engineering. I'm optimistic that we'll have more results in the future.

Part III

Now let's go on to Part III, where I'd like to look at what the impact of all this might be on software engineering. But first let's think about what the prospects are for practical results. I'd like to do this by analogy. If you look back at the history of rule-based systems, the first ones were built around 1970, for example, Mycin [Shortliffe, 1976]. The first, what I would call solid, practical use of the technology was around 1980 with XCON (then called R1) [McDermott, 1981], and depending on whose publicity you look at, it seems they're in widespread use, and have been for a couple of years. So let's say that there's something like a 15-year gap between experimental systems and widespread use.

Now, in the application of AI techniques to software engineering, if we could have started spanning that 15-year gap last year, we ought to have widespread use in 2001. For programming-in-the-small, I'm actually a little bit more optimistic than that. I think we are close to having some experimental systems, things like Φ NIX that we're working on, or the REFINE system, or work at ISI [Balzer, 1985], AT&T [Kelley, 1987], Mitre [Brown, 1985], and several other places [Prywes et al., 1979]. Any one of these may turn out to be a good experimental system in a practical situation. I think there's good hope of having some actual use in practical situations sometime in the 1990s. If so, maybe before the next century we will have relatively widespread use of programming support systems based on AI for programming-in-the-small.

I'm considerably more pessimistic about programming-in-the-large. I think the earliest that we can hope for any kind of experimental system in practical situations is the 1990s, and I hope I'm not being too optimistic in saying that. I think the issues are just extremely difficult, and that's why I want to stress again that there is a single issue on which, if we could marshal a whole lot of work on it, we might make some progress; that is, how do you capture the design of a large system? And it's that issue that I think is either going to make or break the work in this area. But I certainly don't expect to see practical systems by earlier than sometime in the next century.

But let's assume success and see what the impact might be. In terms of the direct impact on software engineering, I think we're likely to gain a factor of two or three in productivity, a few hundred percent, based on some simple back-of-the-envelope calculations. Figure 9a shows what the impact of having an automatic programming system might be on programming-in-the-small. On

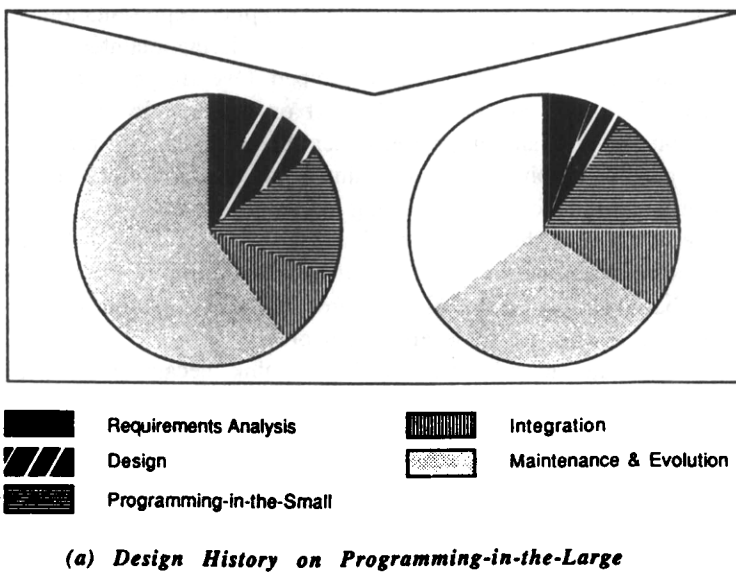
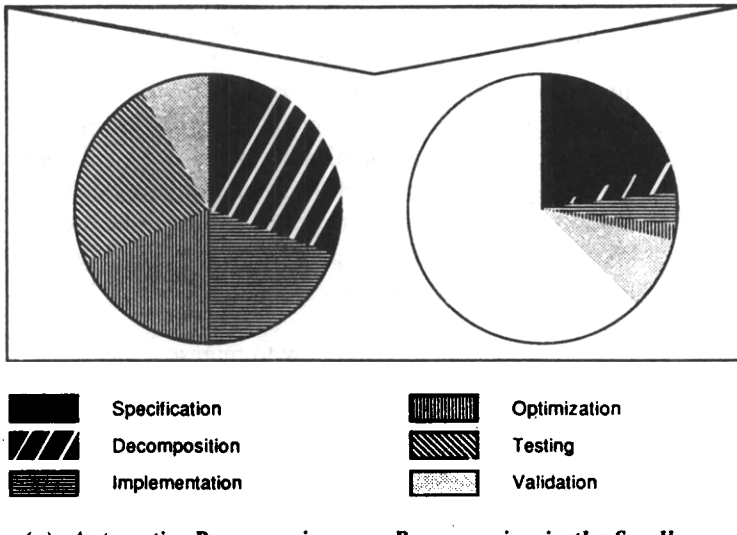


Figure 9 Direct impact of artificial intelligence on software engineering activities.

the left is the same pie chart I showed you earlier. On the right is what I think the effort might be if you had one of these hypothetical automatic programming systems. One of the interesting things to see is that the specification would probably be harder, and would take more effort, because you wouldn't be able to get by with inadequate specifications. You'd be forced to make sure they're adequate. So that would probably take more work. On the other hand, decomposition and implementation and optimization ought to be substantially helped. Testing really ought to go away—if there's one thing we ought to buy out of this, it's to get away from the need for testing. But validation, that is, making sure you have the right program and the specifications are right, is not likely to be affected at all. So we can see here a factor of two or three or so overall.

Now, let's suppose someone's figured out how to represent the design history of a large system, what might that do? Here again, you see the pie chart I showed earlier. Requirements analysis is not likely to be affected at all. Design might actually be speeded up, because a lot of design is a collaborative effort, and if you have a formal representation of the thing you're designing, that ought to help the communication process. So we can guess that design might be a little bit easier. The programming-in-the-small part is not going to be helped at all. Integration is probably also not going to be helped at all. Maintenance might be cut in half, because you have an explicit representation of all those decisions that are the focus of your attention during maintenance and evolution. So that's an area that ought to be helped. Overall, you can see that you've gained a factor of about one-and-a-half on productivity. If you also throw in an automatic programming system, then the programming-in-the-small piece ought to go down a bit, as well as maintenance and evolution. So you'll find again, that you get a factor of something like two or three.

Now, when I first put these together, I was rather disappointed. I wanted to be able to say that you could get orders of magnitude improvement in productivity. And based simply on this kind of direct effect, I think that's not true. If we think that's true, then we're deceiving ourselves.

However, the situation isn't quite that bad, I think, because there will be indirect effects of having a sophisticated technology like this helping software engineering. One indirect effect is that you'll be able to look at programs that are now in the programming-in-the-large area, but think of them as programming-in-the-small. And the effort involved in programming-in-the-small is substantially less, so we ought to gain something there just because we can think of some large problems as being small problems.

But more interestingly, I think the successful application of AI techniques to software engineering activities will enable the use of profoundly different software development paradigms. And this is where I think the real gain in all of this is going to be.

Let's look at a couple of examples. Let's think about what automatic programming would do for rapid prototyping. There's a lot of emphasis now on building rapid prototypes of software systems in order to understand the requirements, to understand what it is that you want the system to do. Not how to build it, but what it is you want it to do. Now, if we have automatic programming systems, we ought to be able to build those prototypes much more rapidly, and therefore do the requirements analysis much more rapidly; and in addition, we ought to be able to convert to a final product more easily because we have a formal representation of the specifications. We don't just have the prototype, we also have the specifications for the prototype. And this in fact is one of the effects that people using REFINE have noted. So there is some indication that the indirect impact may actually be happening.

As another example, think of what a design history would do for software reuse. I mentioned before that one problem in software reuse is finding the thing you want to reuse. The other problem is that it isn't quite what you want. It may be very close, but not quite. And what not quite means is that some implementation decisions were not made correctly, at least from your point of view. There may have been hundreds or thousands of decisions made along the way, but some of them, a few of them, are not the ones that you would have made if you were writing the software in the first place. Now, normally that means that you can't reuse the software, or you have to go through a great deal of evolution on that software. But if we had a representation of the design history that allowed us to replay most of the development of that software automatically, then all we would have to worry about are those few decisions that were made in the wrong way. And therefore, it ought to be much easier to reuse software that is similar, but not identical, to what is required.

3.1 Review

So let's review Part III. I think that practical results for programming-in-the-small ought to be achievable in the 1990s, and there's a lot of work now that ought to get us there. But for programming-in-the-large, I think the practical results are really very, very far away, not before the turn of the century. Finally, the real impact of AI will not be directly on software engineering in terms of automating or supporting specific individual activities, but rather, indirectly, by letting us use profoundly different paradigms.

4 Conclusion

Now let me pop up one more level and summarize what I've tried to say. First, software engineering activities are knowledge intensive. They require substantial

knowledge of the application domain and the target software itself, and AI techniques ought to be able to help manage that knowledge effectively. But unfortunately, past research has not demonstrated practical success in this area, in part because of the amount and diversity of the knowledge, and in part because the efforts have been relatively narrowly focused. For programming-in-the-small, I think the time is right for substantial experiments in practical situations. For programming-in-the-large, I think there are several major research issues, most importantly, how to represent the design of large systems, that have to be addressed before there will be any practical demonstrations. And finally, the real long-term impact of AI will not be on the individual activities, but rather on the software development paradigms that it will let us use.

Note

For other discussions of the relationship between AI and SE, see Simon's presentation at the Seventh International Conference on Software Engineering [Simon, 1986], Brooks's presentation at the 1986 International Federation of Information Processing Conference [Brooks, 1987], the special issue of *IEEE Transactions on Software Engineering* [Mostow, 1985], the report on the Knowledge-Based Software Assistant [Green et al., 1983], and several collections of papers [Barstow et al., 1984; Biermann et al., 1984; Rich and Waters, 1986].

Acknowledgments

This paper is an edited transcript of a presentation at the Sixth National Conference on Artificial Intelligence, Seattle, Washington, July 17, 1987; based in part on presentations at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, October 30, 1986, and at the Ninth International Conference on Software Engineering, Monterey, California, April 1, 1987. The presentation and paper have benefited substantially from suggestions from Bob Balzer, Paul Barth, Barry Boehm, Steve Fickas, Cordell Green, Sol Greenspan, Elaine Kant, and Stan Vestal.

References

Association for Computing Machinery, 1986. *Proceedings of the Conference on Computer-Supported Cooperative Work*, Austin, Texas.

- Balzer, R., N. Goldman, and D. Wile, 1978. Informality in program specifications. *IEEE Transactions on Software Engineering* 4(1):94–103. Reprinted in Rich and Waters, 1986.
- Balzer, R., 1981. Transformational programming: An example. *IEEE Transactions on Software Engineering* 7(1):3–14.
- Balzer, R., D. Cohen, M. Feather, N. Goldman, W. Swartout, and D. Wile, 1983. Operational specification as the basis for specification validation. *Theory and Practice of Software Technology*, Ferrari, Bolognani, and Goguen, ed. North Holland, Amsterdam.
- Balzer, R., 1985. A 15 year perspective on automatic programming. *IEEE Transactions on Software Engineering* 11(11):1257–1268.
- Balzer, R., 1987. Living in the next-generation operating system. *IEEE Software* 4(6):77–85.
- Barstow, D., 1979. An experiment in knowledge-based automatic programming. *Artificial Intelligence* 12(2):73–119. Reprint Rich and Waters, 1986.
- Barstow, D., 1982. The roles of knowledge and deduction in algorithm design. *Machine Intelligence 10*, J. Hayes, D. Michie, and Y.-H. Pao, ed. Ellis Horwood and Wiley, New York. Reprinted in Biermann et al., 1984.
- Barstow, D., 1984. A perspective on automatic programming. *AI Magazine* 5(1):5–27.
- Barstow, D., H. Shrobe, and E. Sandewall, ed., 1984. *Interactive Programming Environments*. McGraw-Hill, New York.
- Barstow, D., 1985a. Automatic programming for streams. In *Proceedings of the Ninth International Joint Conference on Artificial Intelligence*, Los Angeles, CA, pp. 232–237. Morgan Kaufmann, San Mateo, California.
- Barstow, D., 1985b. Domain-specific automatic programming. *IEEE Transactions on Software Engineering* 11(11):1321–1336.
- Bauer, F., M. Broy, W. Dosch, R. Gnatz, B. Krieg-Brückner, A. Laut, M. Luckmann, T. Matzner, B. Möller, H. Partsch, P. Pepper, K. Samelson, R. Steinbrüggen, M. Wirsing, and H. Wössner, 1978. Programming in a wide spectrum language: A collection of examples. *Science of Computer Programming* 1:73–114.
- Bauer, F., M. Broy, W. Dosch, F. Geiselbrechtinger, W. Hesse, R. Gnatz, B. Krieg-Brückner, A. Laut, T. Matzner, B. Möller, F. Nickl, H. Partsch, P. Pepper, K. Samelson, M. Wirsing, and H. Wössner, 1982. *Algorithmic Language and Program Development*. Springer-Verlag, New York.
- Biermann, A., G. Guiho, and Y. Kodratoff, ed., 1984. *Automatic Program Construction Techniques*. Macmillan, New York.
- Boehm, B., 1981. *Software Engineering Economics*. Prentice-Hall, Englewood Cliffs, New Jersey.
- Borgida, A., S. Greenspan, and J. Mylopoulos, 1985. Knowledge representation as the basis for requirements specifications. *IEEE Computer* 18(4):82–91. Reprinted in Rich and Waters, 1986.

- Brooks, F. P., 1987. No silver bullet: Essence and accidents of software engineering. *IEEE Computer* 20(4):10–19.
- Brown, R., 1985. Automation of programming: The ISFI experiments. Technical Report M85-21, MITRE, Bedford, Massachusetts.
- Burstall, R. and J. Darlington, 1977. A transformation system for developing recursive programs. *Journal of the ACM* 24(1):44–67.
- Cheatham, T., J. Townley, and G. Holloway, 1979. A system for program refinement. In *Fourth International Conference on Software Engineering*, Munich, Germany, pp. 53–62. Reprinted in Barstow et al., 1984.
- DeRemer, F. and H. Kron, 1976. Programming in the large versus programming in the small. *IEEE Transactions on Software Engineering* 2(2):80–86.
- Dijkstra, E. W., 1976. *A Discipline of Programming*. Prentice-Hall, Englewood Cliffs, New Jersey.
- Fickas, S., 1985a. Automating the transformational development of software. *IEEE Transactions on Software Engineering* 11(11):1268–1277.
- Fickas, S., 1985b. A knowledge-based approach to specification acquisition and construction. Technical Report CIT-TR 85-13, Department of Computer Science, University of Oregon.
- Floyd, R., 1967. Assigning meaning to programs. *Mathematical Aspects of Computer Science*, J. Schwartz, ed., pp. 19–32. American Mathematical Society.
- Green, C., 1969. Application of theorem proving to problem solving. In *Proceedings of the First International Joint Conference on Artificial Intelligence*, Washington, D.C., pp. 219–239. Morgan Kaufmann, San Mateo, California.
- Green, C., 1976. The design of the program synthesis system. In *Second International Conference on Software Engineering*, San Francisco, pp. 4–18.
- Green, C., D. Luckham, R. Balzer, T. Cheatham, and C. Rich, 1983. Report on a knowledge-based software assistant. Technical Report, Kestrel Institute. Reprinted in Rich and Waters, 1986.
- Heidorn, G., 1976. Automatic programming through natural language dialogue: A survey. *IBM Journal of Research and Development* 20(4):302–313. Reprinted in Rich and Waters, 1986.
- Hoare, C. A. R., 1969. An axiomatic basis for computer programming. *Communications of the ACM* 12(10):576–583.
- Johnson, W. and E. Soloway, 1985. PROUST: Knowledge-based program understanding. *IEEE Transactions on Software Engineering* 11(3):267–275. Reprinted in Rich and Waters, 1986.
- Kant, E. and D. Barstow, 1981. The refinement paradigm: The interaction of coding and efficiency knowledge in program synthesis. *IEEE Transactions on Software Engineering* 7(5):458–471. Reprinted in Barstow et al., 1984.
- Kant, E., 1983. On the efficient synthesis of efficient programs. *Artificial Intelligence Journal* 20(3):253–306. Reprinted in Rich and Waters, 1986.

- Kant, E., 1985. Understanding and automating algorithm design. *IEEE Transactions on Software Engineering* 11(11):1361–1374.
- Katz, S. and R. Zimmerman, 1981. An advisory system for developing data representations. In *Proceedings of the Seventh International Joint Conference on Artificial Intelligence*, Vancouver, British Columbia, Canada, pp. 1030–1036. Morgan Kaufmann, San Mateo, California.
- Kelley, V. E., 1987. Inferring formal software specifications from episodic descriptions. In *Proceedings of the Sixth National Conference on Artificial Intelligence*, Seattle, Washington, 127–132. Morgan Kaufmann, San Mateo.
- King, J. C., 1971. Proving programs to be correct. *IEEE Transactions on Computers* 20(11).
- Manna, Z. and R. Waldinger, 1980. A deductive approach to program synthesis. *ACM Transactions on Programming Languages and Systems* 2(1):90–121. Reprinted in Rich and Waters, 1986.
- Manna, Z. and R. Waldinger, 1985. The origin of the binary-search paradigm. In *Proceedings of the Ninth International Joint Conference on Artificial Intelligence*, Los Angeles, CA, pp. 222–224. Morgan Kaufmann, San Mateo, California.
- McDermott, J., 1981. R1: The formative years. *AI Magazine* 2(2):21–29.
- Mostow, J., ed., 1985. Special issue on artificial intelligence and software engineering. *IEEE Transactions on Software Engineering* 11(11).
- Partsch, H. and R. Steinbrüggen, 1983. Program transformation systems. *Computing Surveys* 15(3):199–236.
- Prywes, N., A. Pnuelli, and S. Shastri, 1979. Use of a nonprocedural specification language and associated program generator in software development. *ACM Transactions on Programming Languages and Systems* 1(2):196–217.
- REFINE Users Guide, 1985. Reasoning Systems, Inc., Palo Alto, California.
- Rich, C. and H. Shrobe, 1978. Initial report on a Lisp programmer's apprenticeship. *IEEE Transactions on Software Engineering* 4(6):456–467. Reprinted in Barstow et al., 1984.
- Rich, C., 1981. A formal representation for plans in the programmer's apprenticeship. In *Proceedings of the Seventh International Joint Conference on Artificial Intelligence*, Vancouver, British Columbia, Canada, pp. 1044–1052. Morgan Kaufmann, San Mateo, California.
- Rich, C. and R. Waters, ed., 1986. *Readings in Artificial Intelligence and Software Engineering*. Morgan Kaufmann, San Mateo, California.
- Shortliffe, E. H., 1976. *Computer-Based Medical Consultations: MYCIN*. Elsevier-North Holland, New York.
- Simon, H., 1986. Whether software engineering needs to be artificially intelligent. *IEEE Transactions on Software Engineering* 12(7):726–732.
- Smith, D., 1984. The synthesis of Lisp programs from examples: A survey. *Automatic Program Construction Techniques*, A. Biermann, G. Guiho, and Y. Kodratoff, ed., pp. 307–324. Macmillan, New York.

- Smith, D., 1985. Top-down synthesis of divide-and-conquer algorithms. *Artificial Intelligence Journal* **27**(1):43–96. Reprinted in Rich and Waters, 1986.
- Smith, D., G. Kotik, and S. Westfold, 1985. Research on knowledge-based software environments at Kestrel Institute. *IEEE Transactions on Software Engineering* **11**(11):1278–1295.
- Stefik, M., G. Foster, D. Bobrow, K. Kahn, S. Lanning, and L. Suchman, 1987. Beyond the chalkboard: Computer support for collaboration and problem solving in meetings. *Communications of the ACM* **30**(1):32–47.
- Steier, D. and E. Kant, 1985. The roles of execution and analysis in algorithm design. *IEEE Transactions on Software Engineering* **11**(11):1374–1386.
- Swartout, W., 1983. Xplain: A system for creating and explaining expert consulting systems. *Artificial Intelligence Journal* **21**(3):285–325.
- Waldinger, R., 1969. PROW: A step toward automatic program writing. In *Proceedings of the First International Joint Conference on Artificial Intelligence*, Washington, D.C., pp. 241–252. Morgan Kaufmann, San Mateo, California.
- Waters, R., 1985. The programmer's apprentice: A session with KBEmacs. *IEEE Transactions on Software Engineering* **11**(11):1296–1320. Reprinted in Rich and Waters, 1986.
- Wile, D., 1983. Program developments: Formal explanations of implementations. *Communications of the ACM* **26**(11):902–911. Reprinted in Rich and Waters, 1986.